

ADC Module of PIC Microcontroller:

The **ADC module in PIC microcontroller** stand for Analog to Digital converter Module. This is an inbuilt component available almost in all **PIC Microcontroller**. In order to read the physical data from real world and process under CPU in the digital form we need Analog to Digital converter. There are many source of physical data values like temperature, pressure, current, voltage, distance sensor which produce an Analog signal in 0-20ma or 0-5V or 0-10V. To read this type of Analog data from real world we need to convert into digital format. ADC module comes to help as we need to process this data under CPU in digital format.

The [Analog-to-Digital Converter \(ADC\)](#) allows conversion of an analog input signal to a 10-bit binary representation of that signal. This device uses analog inputs, which are multiplexed into a single sample and hold circuit. The output of the sample and hold is connected to the input of the converter. The converter generates a 10-bit binary result via successive approximation and stores the conversion result into the ADC result registers.

In **PIC Controller PIC16F877A** the **ADC voltage reference is software selectable** to be either internally generated or externally supplied. The A/D converter has a unique feature of being able to operate while the device is in Sleep mode. To operate in Sleep, the A/D clock must be derived from the A/D's internal RC oscillator.

The A/D module has four registers. These registers are:

- A/D Result High Register (ADRESH)
- A/D Result Low Register (ADRESL)
- A/D Control Register 0 (ADCON0)
- A/D Control Register 1 (ADCON1)

ADC Configuration:

When configuring and using the ADC the following functions must be considered:

- Configure analog pins/voltage reference and digital I/O (ADCON1)
- Select A/D conversion clock (ADCON0)
- Select A/D conversion clock (ADCON0)
- Turn on A/D module (ADCON0)
- ADC conversion clock source
- Interrupt control

Configure A/D interrupt (if desired):

— Clear ADIF bit

Set ADIE bit

— Set PEIE bit

— Set GIE bit

Channel selection

The CHS bits of the ADCON0 register determine which channel is connected to the sample and hold circuit. When changing channels, a delay is required before starting the next conversion.

ADCON0 REGISTER (ADDRESS 1Fh)

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	U-0	R/W-0
ADCS1	ADCS0	CHS2	CHS1	CHS0	GO/ $\overline{\text{DONE}}$	—	ADON
bit 7							bit 0

bit 5-3 CHS2:CHS0: Analog Channel Select bits

000 = Channel 0 (AN0)

001 = Channel 1 (AN1)

010 = Channel 2 (AN2)

011 = Channel 3 (AN3)

100 = Channel 4 (AN4)

101 = Channel 5 (AN5)

110 = Channel 6 (AN6)

111 = Channel 7 (AN7)

The ADCON1 register used to configure the functions of the port pins. The port pins can be configured as analog inputs (RA3 can also be the voltage reference) or as digital I/O.

bit 5-3 **CHS2:CHS0:** Analog Channel Select bits

000 = Channel 0 (AN0)

001 = Channel 1 (AN1)

010 = Channel 2 (AN2)

011 = Channel 3 (AN3)

100 = Channel 4 (AN4)

101 = Channel 5 (AN5)

110 = Channel 6 (AN6)

111 = Channel 7 (AN7)

ADC voltage reference selection

The ADPREF bits of the ADCON1 register provides control of the positive voltage reference.
The positive voltage reference can be:

- VREF+ pin
- VDD

ADC conversion clock source

The source of the conversion clock is software selectable via the ADCS bits of the ADCON1 register. There are seven possible clock options:

- FOSC/2
- FOSC/4
- FOSC/8
- FOSC/16
- FOSC/32
- FOSC/64
- FRC (dedicated internal FRC oscillator)

The time to complete one bit conversion is defined as TAD. One full 10-bit conversion requires 11.5 TAD periods.

Interrupt control

The ADC module allows for the ability to generate an interrupt upon completion of an Analog-to-Digital conversion. The ADC Interrupt Flag is the ADIF bit in the PIR1 register. The ADC Interrupt Enable is the ADIE bit in the PIE1 register. The ADIF bit must be cleared in software.

Result formatting

The 10-bit ADC conversion result can be supplied in two formats, left justified or right justified. The ADFM bit of the ADCON1 register controls the output format.

ADC Acquisition Requirements

The source impedance (RS) and the internal sampling switch (RSS) impedance directly affect the time required to charge the capacitor CHOLD. The sampling switch (RSS) impedance varies over the device voltage (VDD). For the ADC to meet its specified accuracy, the charge holding capacitor (CHOLD) must be allowed to fully charge to the input channel voltage level. The source impedance (RS) and the internal sampling switch (RSS) impedance directly affect the time required to charge the capacitor CHOLD. The sampling switch (RSS) impedance varies over the device voltage (VDD).

The maximum recommended impedance for analog sources is 10 k Ω . As the source impedance is decreased, the acquisition time may be decreased. After the analog input channel is selected (or changed), an ADC acquisition must be done before the conversion can be started. To calculate the minimum acquisition time, Equation 16-1 may be used. This equation assumes that 1/2 LSb error is used (1,024 steps for the ADC). The 1/2 LSb error is the maximum error allowed for the ADC to meet its specified resolution.

ACD Control Register

R/W-0/0	R/W-0/0	R/W-0/0	R/W-0/0	U-0	U-0	R/W-0/0	R/W-0/0
ADFM	ADCS<2:0>			—	—	ADPREF<1:0>	
bit 7				bit 0			

Legend:		
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'
u = Bit is unchanged	x = Bit is unknown	-n/n = Value at POR and BOR/Value at all other Resets
'1' = Bit is set	'0' = Bit is cleared	

bit 7	ADFM: ADC Result Format Select bit 1 = Right justified. Six Most Significant bits of ADRESH are set to '0' when the conversion result is loaded. 0 = Left justified. Six Least Significant bits of ADRESL are set to '0' when the conversion result is loaded.
bit 6-4	ADCS<2:0>: ADC Conversion Clock Select bits 000 = Fosc/2 001 = Fosc/8 010 = Fosc/32 011 = FRC (clock supplied from a dedicated FRC oscillator) 100 = Fosc/4 101 = Fosc/16 110 = Fosc/64 111 = FRC (clock supplied from a dedicated FRC oscillator)
bit 3-2	Unimplemented: Read as '0'
bit 1-0	ADPREF<1:0>: ADC Positive Voltage Reference Configuration bits 00 = VREF is connected to VDD 01 = Reserved 10 = VREF is connected to external VREF+ pin ⁽¹⁾

ADC CONVERSION PROCEDURE

Follow the procedure for using the ADC to perform an Analog-to-Digital conversion:

1. Configure Port:

- Disable pin output driver (Refer to the TRIS register)
- Configure pin as analog (Refer to the ANSEL register)
- Disable weak pull-ups, either globally (Refer to the OPTION_REG register) or individually (Refer to the appropriate WPUx register)

2. Configure the ADC module:

- Select ADC conversion clock
- Configure voltage reference
- Select ADC input channel
- Turn on ADC module

3. Configure ADC interrupt (optional):

- Clear ADC interrupt flag
- Enable ADC interrupt
- Enable peripheral interrupt
- Enable global interrupt⁽¹⁾

4. Wait the required acquisition time⁽²⁾.

5. Start conversion by setting the GO/DONE bit.

6. Wait for ADC conversion to complete by one of the following:

- Polling the GO/DONE bit
- Waiting for the ADC interrupt (interrupts enabled)

7. Read ADC Result.

8. Clear the ADC interrupt flag (required if interrupt is enabled)

To read more details about ADC under PIC PIC16F877A please keep [PIC16F877A datasheet](#) with you.

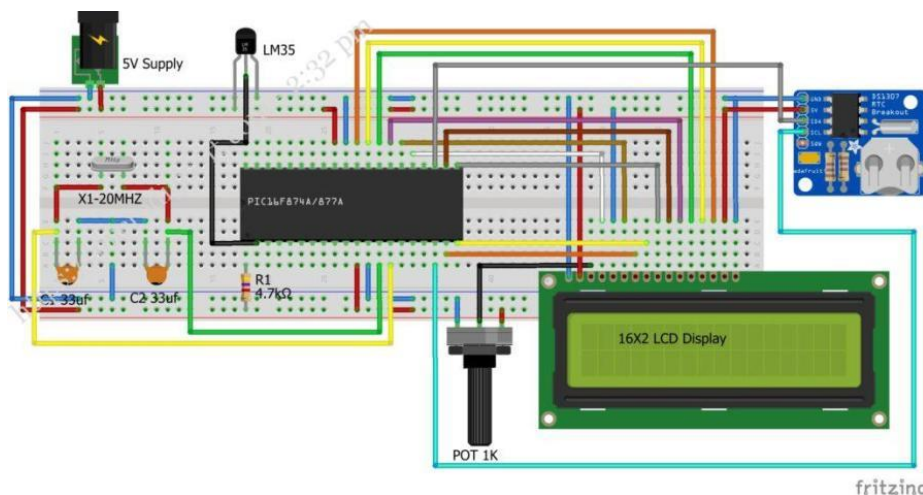
Example: Reading Temperature using ADC module

Let us take simple example to read temperature sensor from LM35 sensor using LCD. To build this project we need LM35 temperature sensor and **16X2 LCD display** to monitor the temperature data.

In the previous tutorial we have seen [Interfacing LCD display with PIC microcontroller](#) and [Interfacing RTC with PIC Microcontroller](#).

Let us take Analog Input from LM35 Sensor on ADC input AN0 from PORTA. As I explain in previous tutorial to getting start with PIC controller programming we need install MPLAB X IDE software and XC8 compiler on our system.

Also we need Proteus simulator to simulate the circuit with program. So let us prepare circuit diagram to **interface LM35 Temperature sensor** to read the value using ADC Module of PIC Microcontroller with MPLAB and XC8.



Program to Upload

/*

```

#include "config.h" // Send PIC 16f877a configuration

#include "lcd.h" //call LCD Functions

#include "I2C.h" // Call I2C Functions

#include "RTC.h" // Call RTC functions

char Tempr[] = " 00.0 C"; //set Temp display format in kelvin

int C;

void ADC_Init()

{

TRISA = 0xff; /*set as input port*/

ADCON1 = 0b11110000; //Right Justify, FRC Clock , All pins as Analog Input and setting
Reference Voltages

ADCON0 = 0b00000001; //Turn ON ADC and Clock Selection

ADRESH = 0; /*Flush ADC output Register*/

ADRESL = 0;

}

unsigned int ADC_Read(unsigned char channel)

{

ADCON0 |= channel << 1; //Setting the required Bits

__delay_ms(2); //Acquisition time to charge hold capacitor

GO_DONE = 1; //Initializes A/D Conversion

```

```

while(GO_nDONE); //Wait for A/D Conversion to complete

return ((ADRESH<< 8)+ADRESL); //Returns Result
}

void main() {

    Init_LCD(); // start LCD function

    InitI2C(); // Initialize I2C bus function

    ADC_Init(); /* initialize 10-bit ADC*/

    while(1)

    {

        Lcd_Clear();

        Lcd_Set_Cursor(0x80);

        Lcd_Write_String(" Welcome ");

        Lcd_Set_Cursor(0xC0);

        Lcd_Write_String(" Microdigisoft ");

        __delay_ms(50);      // 50ms Delay.

        Lcd_Clear();

        Lcd_Set_Cursor(0x80);

        DisplayDateOnLCD(Get_DS1307_RTC_Date());

        Lcd_Set_Cursor(0);

        C = ADC_Read(0) * 0.489;      // // Read analog voltage and convert it to degree Celsius
        (0.489 = 500/1023)
    }
}

```

```
if (C > 99)

    Tempr[0] = 1 + 48;          // Put 1 (of hundred)

else

    Tempr[0] = ' ';           // Put space

    Tempr[1] = (C / 10) % 10 + 48;

    Tempr[2] = C % 10 + 48;

    Tempr[5] = 223;           // Put degree symbol ( ° )

    Lcd_Set_Cursor(0xC0);

    Lcd_Write_String("Temp:");

    Lcd_Write_String(Tempr);

    __delay_ms(200); //100 ms delay

}

}
```